

Question No:6

Question description

Your task is to maintain an array of n values and efficiently process the following types of queries:

1. Increase the first value in range $[a,b]$ by 1, the second value by 2, the third value by 3, and so on.
2. Calculate the sum of values in range $[a,b]$.

Input

The first input line has two integers n and q : the size of the array and the number of queries.

The next line has n values $t_1, t_2, \dots, t_n$: the initial contents of the array.

Finally, there are q lines describing the queries. The format of each line is either "1 a b" or "2 a b".

Output

Print the answer to each sum query.

Constraints

- $1 \leq n, q \leq 2 \cdot 10^5$
- $1 \leq t_i \leq 10^6$
- $1 \leq a \leq b \leq n$

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 3
3 7 2 2 5
3 2 3
1 3 6
1 2 3
```

EXPECTED OUTPUT

9

Test Case 2

INPUT (STDIN)

```
5 3
4 2 3 1 7
2 1 5
1 1 5
2 1 5
```

EXPECTED OUTPUT

17
32

Code:

```
1  #include <stdio.h>
2
3  #define MAXN 200005
4
5  long long tree[4 * MAXN];
6  long long lazyA[4 * MAXN];
7  long long lazyB[4 * MAXN];
8
9
10 long long sumIndex(int l, int r)
11 {
12     long long n = r - l + 1;
13
14     return (long long)(l + r) * n / 2;
15 }
16
17 void build(int node, int l, int r, long long arr[])
18 {
19     if (l == r)
20     {
21         tree[node] = arr[l];
22         return;
23     }
24
25     int mid = (l + r) / 2;
26
27     build(node * 2, l, mid, arr);
28     build(node * 2 + 1, mid + 1, r, arr);
```

```

30     tree[node] = tree[node * 2] + tree[node * 2 + 1];
31 }
32
33 void apply(int node, int l, int r, long long A, long long B)
34 {
35     /*
36      Add:
37
38      A*i + B
39
40      to every position i from l to r.
41     */
42
43     tree[node] += A * sumIndex(l, r)
44                 + B * (r - l + 1);
45
46     lazyA[node] += A;
47     lazyB[node] += B;
48 }
49
50 void push(int node, int l, int r)
51 {
52     if (lazyA[node] == 0 && lazyB[node] == 0)
53         return;
54
55     if (l != r)

```

```
56     {
57         int mid = (l + r) / 2;
58
59         apply(node * 2,
60             l,
61             mid,
62             lazyA[node],
63             lazyB[node]);
64
65         apply(node * 2 + 1,
66             mid + 1,
67             r,
68             lazyA[node],
69             lazyB[node]);
70     }
71
72     lazyA[node] = 0;
73     lazyB[node] = 0;
74 }
75
76 void update(int node, int l, int r,
77             int ql, int qr,
78             long long A, long long B)
79 {
80     if (qr < l || r < ql)
81         return;
```

```
83     if (ql <= l && r <= qr)
84     {
85         apply(node, l, r, A, B);
86         return;
87     }
88
89     push(node, l, r);
90
91     int mid = (l + r) / 2;
92
93     update(node * 2,
94           l,
95           mid,
96           ql,
97           qr,
98           A,
99           B);
100
101     update(node * 2 + 1,
102           mid + 1,
103           r,
104           ql,
105           qr,
106           A,
107           B);
108 }
```

```
109     tree[node] = tree[node * 2] + tree[node * 2 + 1];
110 }
111
112 long long query(int node, int l, int r,
113                int ql, int qr)
114 {
115     if (qr < l || r < ql)
116         return 0;
117
118     if (ql <= l && r <= qr)
119         return tree[node];
120
121     push(node, l, r);
122
123     int mid = (l + r) / 2;
124
125     return query(node * 2,
126                 l,
127                 mid,
128                 ql,
129                 qr)
130         +
131         query(node * 2 + 1,
132             mid + 1,
133             r,
134             ql,
```

```

135         qr);
136     }
137
138     int main()
139     {
140         int n, q;
141
142         scanf("%d %d", &n, &q);
143
144         long long arr[MAXN];
145
146         for (int i = 1; i <= n; i++)
147         {
148             scanf("%lld", &arr[i]);
149         }
150
151         build(1, 1, n, arr);
152
153         while (q--)
154         {
155             int type, a, b;
156
157             scanf("%d %d %d", &type, &a, &b);
158
159             if (type == 1)
160             {
161                 update(1,
162                     1,
163                     n,
164                     a,
165                     b,
166                     1,
167                     1 - a);
168             }
169             else if (type == 2 || type == 3)
170             {
171                 printf("%lld\n",
172                     query(1,
173                         1,
174                         n,
175                         a,
176                         b));
177             }
178         }
179         return 0;
180     }

```

Output:

Test Case-1:

	5 3	
	3 7 2 2 5	
	3 2 3	
	9	
	1 3 6	
	1 2 3	

Test Case-2:

	5 3	
	4 2 3 1 7	
	2 1 5	
	17	
	1 1 5	
	2 1 5	
	32	